

SOFTWAREDESIGN EN -KWALITEIT

MODELLEREN
BINNEN EEN
APPLICATIEARCHITECTUUR

avans
hogeschool



NA DEZE LES ...

1. Kun je een softwareontwerp onderbrengen binnen een applicatiearchitectuur.
2. Weet je wat architectural constraints zijn
3. Kun je een domeinmodel opstellen

UITWERKING HUISWERK

Voorbeeld uitwerking als demonstratie

HET DOMEINMODEL



Een applicatie maak je in een bepaalde context



Bijvoorbeeld: een bibliotheekreserveringssysteem



In het domein gaat het om de essentiële informatie en bijbehorende regels voor die context



Het vormt het hart van het systeem



Het domein modelleer je d.m.v. een UML klassendiagram

OPDRACHT: DOMEINKLASSEN

❑ Bedenk domeinklassen en relaties voor een bibliotheek



KLASSENDIAGRAM:

OPDRACHT: BIBLIOTHEEKAPPLICATIE

Download het bibliotheek project van Brightspace:

Library code Les2

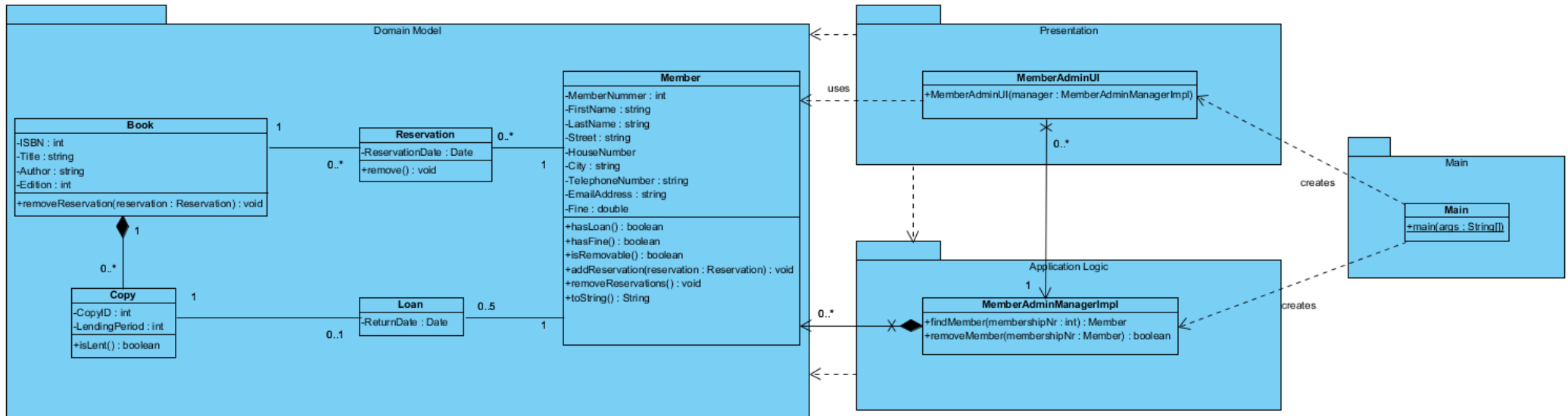
- ☐ Run de code en bekijk de applicatie
- ☐ Bekijk de code en pas jullie UML aan



APPLICATIEARCHITECTUUR:

Architect/Avans Hogeschool

Getters and Setters left out for readability



ANDERE CATEGORIEËN KLASSEN

☐ Gebruikersinteractie

- Grafische User Interface
- Command line

☐ Persistency (permanente opslag in bv. Database, file, Cloud)

☐ ‘Applicatie logica’

- wat is dat?

Een applicatie kan uit meer onderdelen bestaan: presentation, application logic, persistency...

Deze zijn geen onderdeel van het domein, maar van de applicatie die erop is gebaseerd

KLASSEN GROEPEREN

Zoals je bestanden op je harde schijf organiseert, zo organiseer je ook klassen van je applicatie **op basis van verantwoordelijkheden**

*Martin Fowler*¹ onderkent drie hoofdlagen

1. Presentation

Voor de interactie met de gebruiker / Representatie van informatie

2. Domain

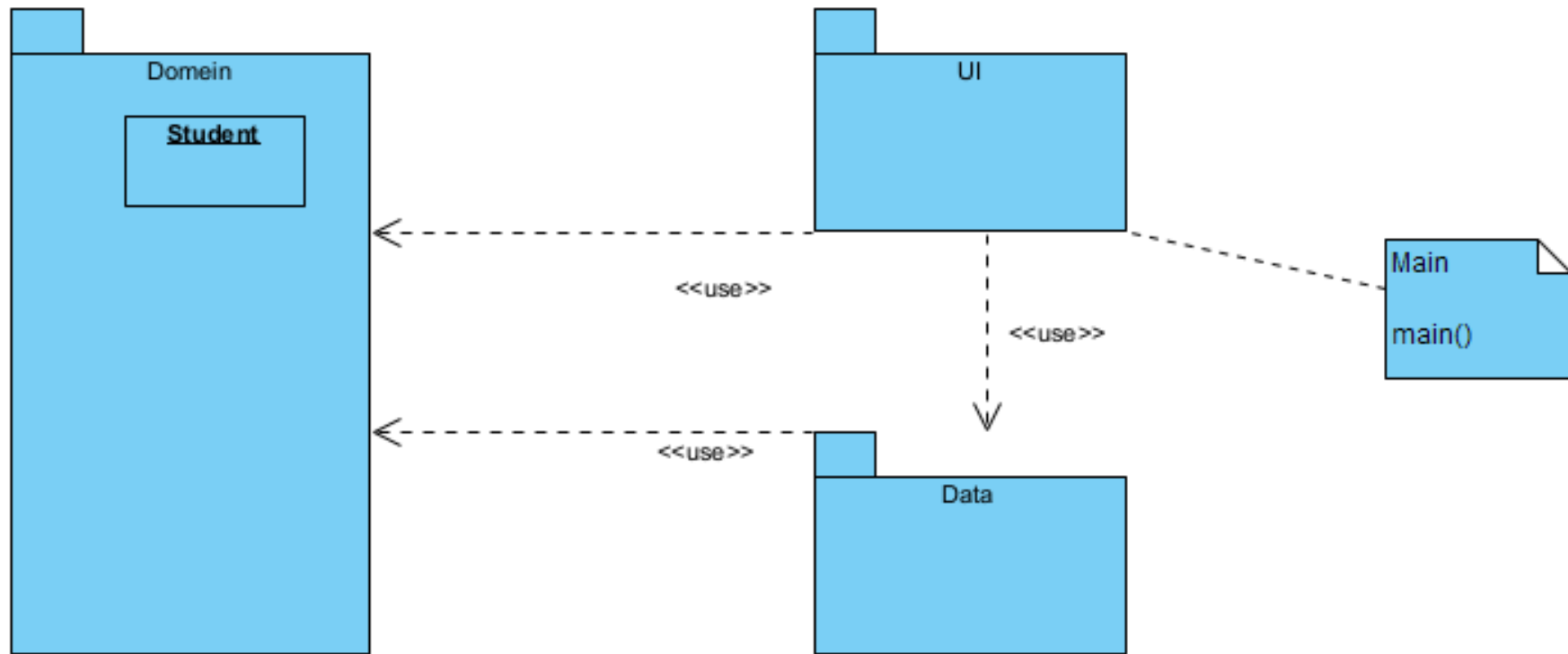
De kern van de applicatie / Essentiële informatie en regels afhankelijk van context

3. Data source / data storage

Bron waar data vandaan komt of opgeslagen wordt, bijv. database, webservice, andere applicatie, etc.

¹: M. Fowler - Patterns of Enterprise Application Architecture

APPLICATIEARCHITECTUUR



KLASSEN GROEPPEREN (2)

Er zijn diverse andere (sub)lagen te bedenken. Wij gaan aan de slag met 4 lagen.

De 4^e laag:

- ☐ Bevat applicatie logica
- ☐ Coördineert taken
- ☐ Zorgt ervoor dat domainklassen geen kennis hebben applicatie logica

Dit maken we zo concreet!

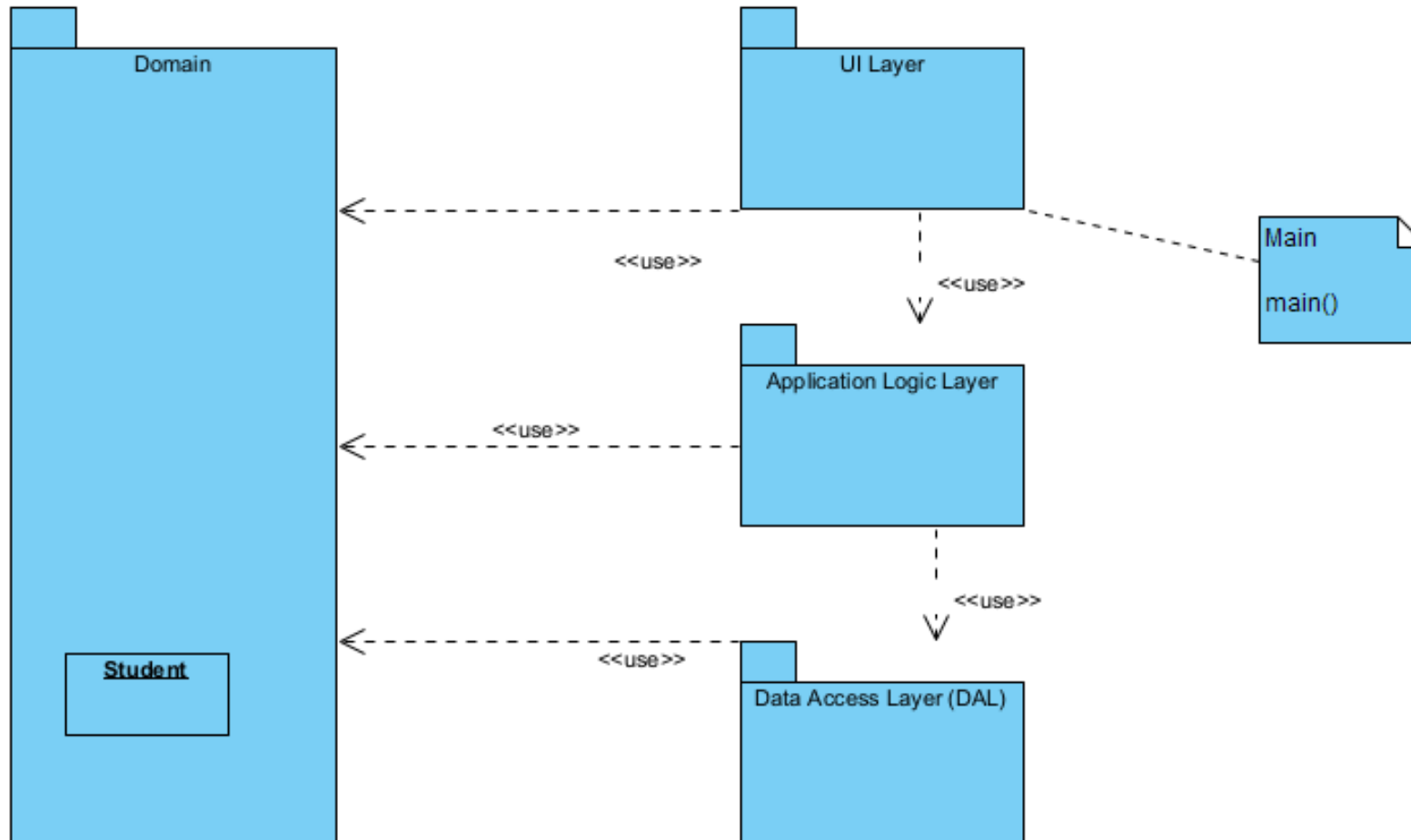
REFERENTIEARCHITECTUUR

- ☐ We gaan werken volgens een **referentiearchitectuur**

- ☐ Referentiearchitectuur:
 - een ontwerp van hoger/abstracter niveau met een aantal basisonderdelen en relaties daartussen

- ☐ Basisonderdelen:
 - Komen overeen met de lagen zoals hiervoor genoemd

REFERENTIEARCHITECTUUR



REFERENTIEARCHITECTUUR

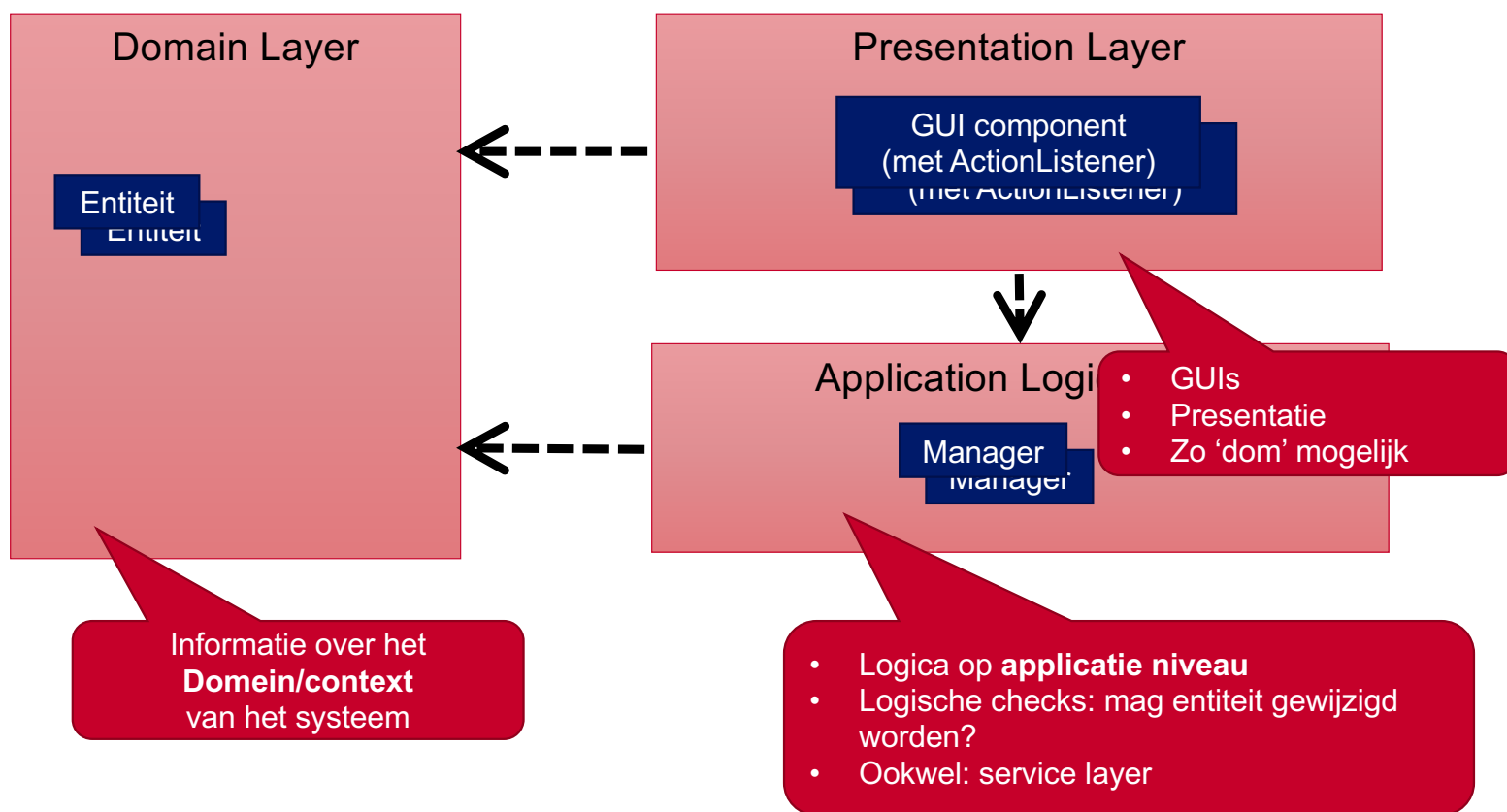
Architect/Avans Hogeschool

Getters and Setters left out for readability



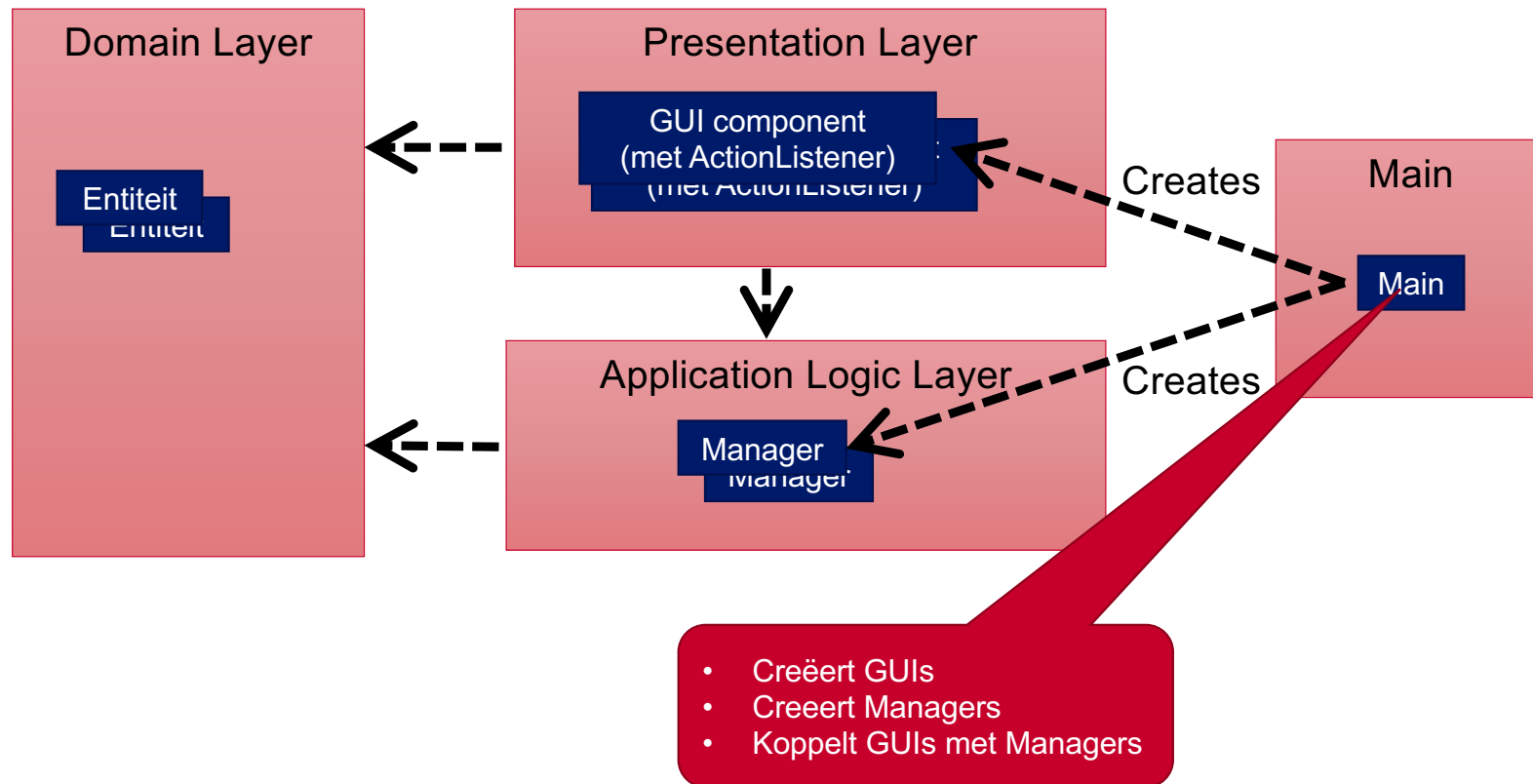
REFERENTIEARCHITECTUUR

ZONDER PERSISTENTIE



REFERENTIEARCHITECTUUR

ZONDER PERSISTENTIE



PERSISTENTIE TOEVOEGEN

☐ Persistent:

- Blijft behouden wanneer applicatie afgesloten is
- Permanent data opslaan in bv database, file, cloud,...

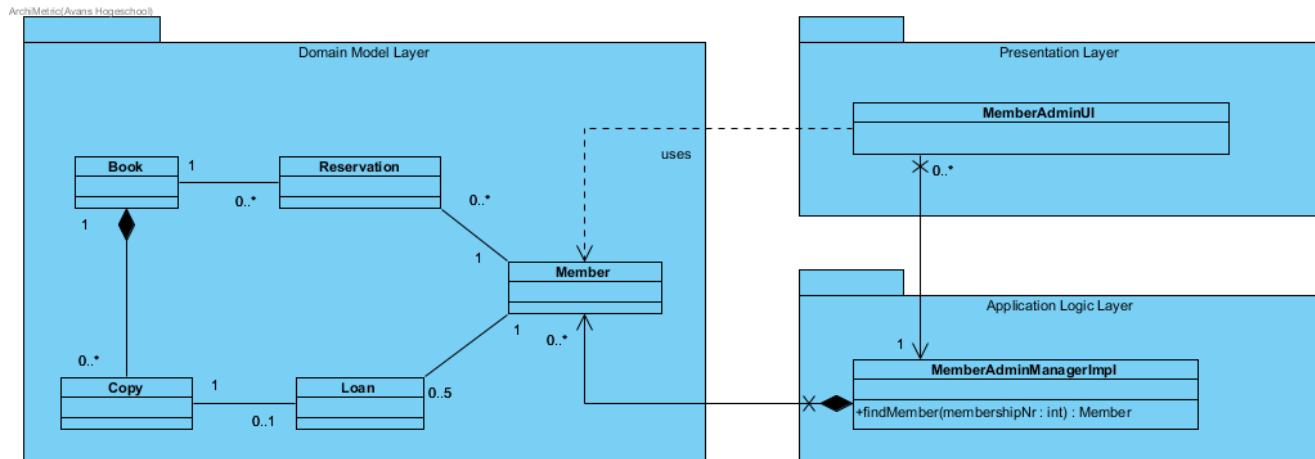
☐ Dat hebben we nog niet in ons ontwerp...

OPDRACHT: PERSISTENTIE TOEVOEGEN

❑ Uitgangspunt: klassendiagram bibliotheek (BS)

❑ Breid het ontwerp uit voor persistentie mbv een relationele database (bv SQLServer)

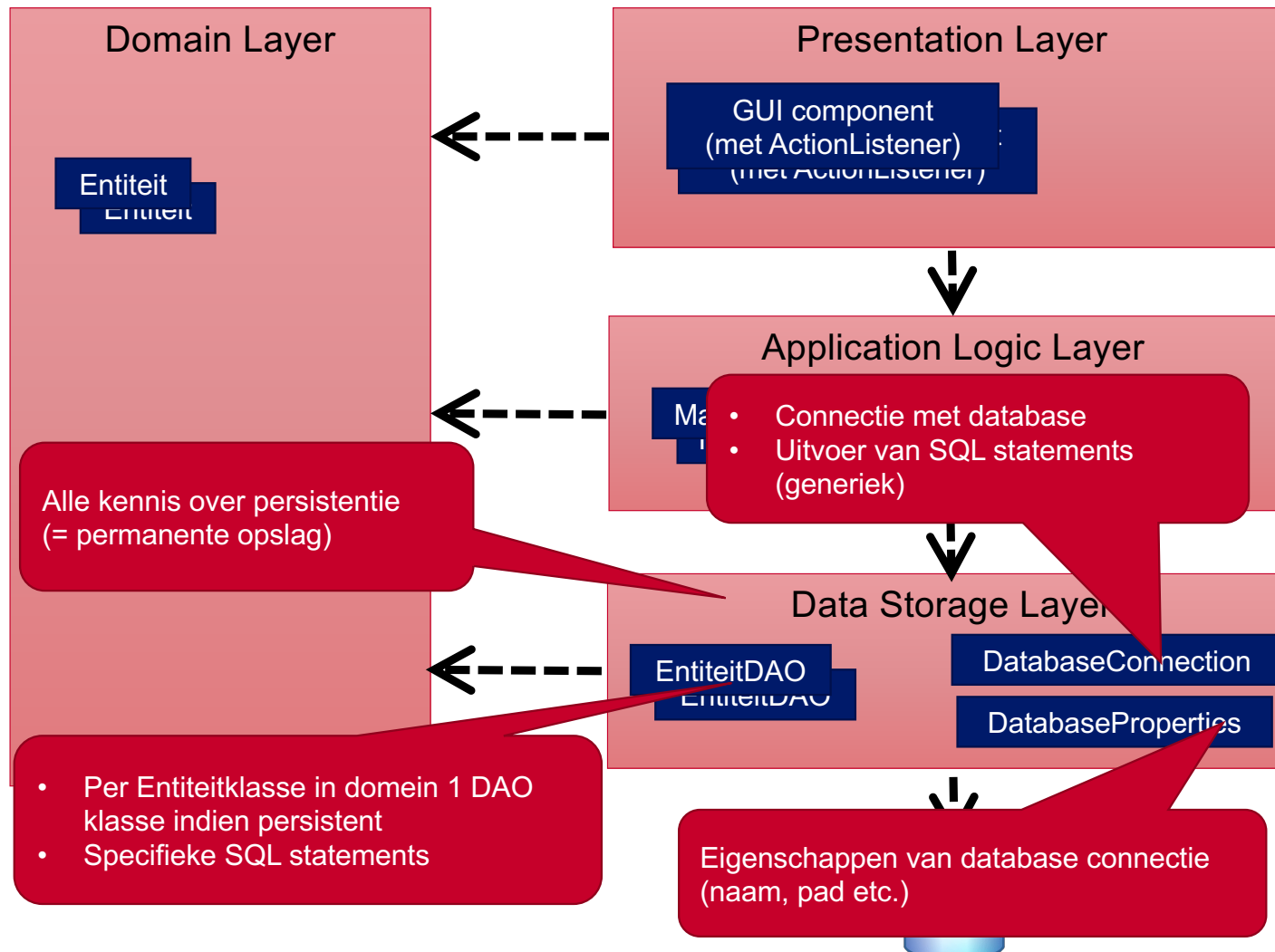
- Hoe pak je dit aan?
- Onderbouwing?



PERSISTENTIE TOEVOEGEN

- ☐ Welke elementen heb je toegevoegd?
- ☐ Waar?
- ☐ Waarom?
- ☐ Waar bevinden zich de SQL queries?
- ☐ Waar bevindt zich overige kennis over het DBMS?

REFERENTIEARCHITECTUUR MET PERSISTENTIE

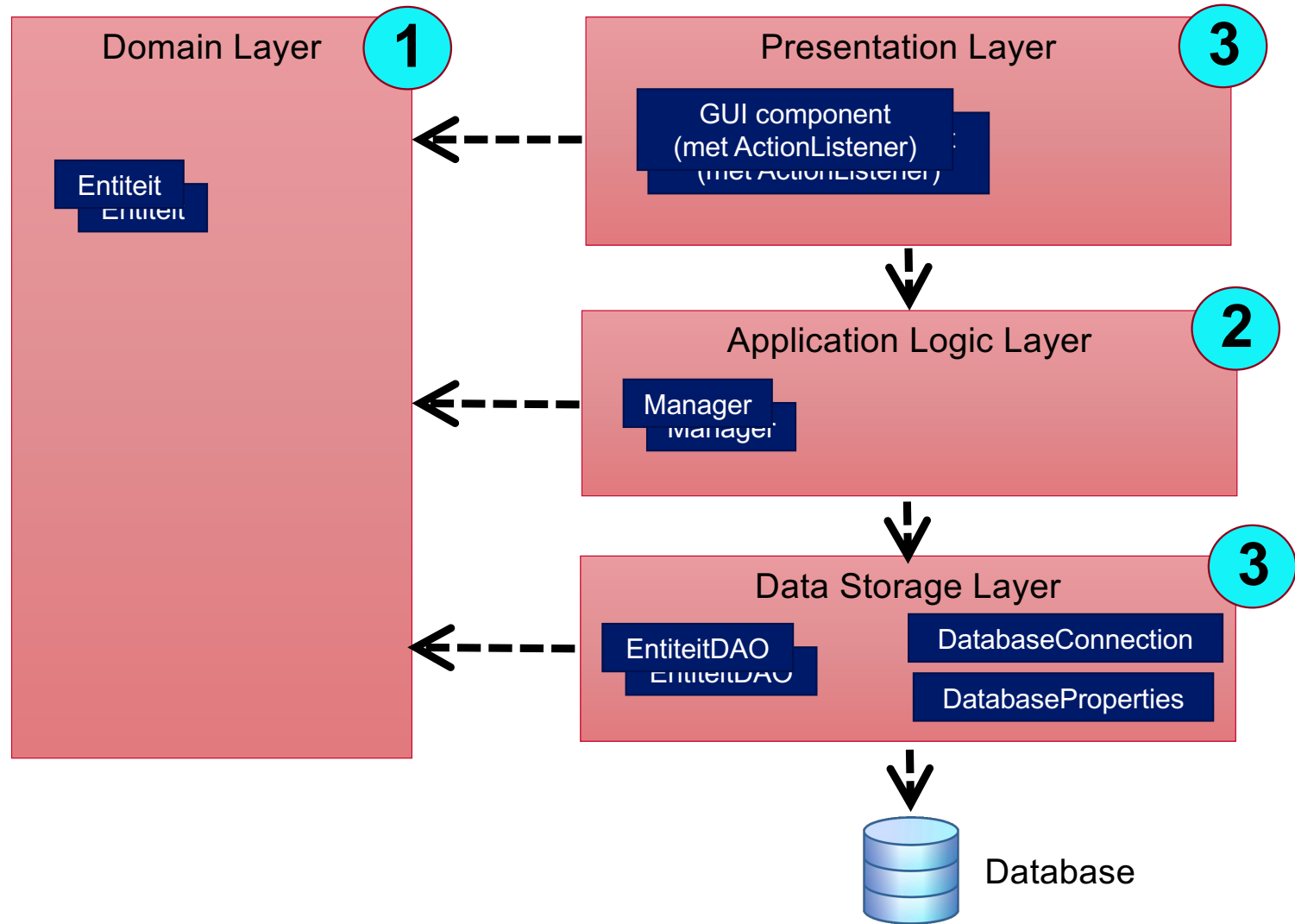


ONTWERPVOLGORDE

1. De domain layer
2. De application logic layer
3. Of de presentation layer óf de data storage layer

Echter je werkt iteratief: telkens een werkende verticale slice met een klein stukje werkende functionaliteit.

ONTWERPVOLGORDE



VERANTWOORDELIJKHEDEN

- ☐ Entiteiten weten alles van het domein, d.w.z. informatie en regels (en verder zo min mogelijk)
- ☐ DAO's weten alles van persistentie
- ☐ Managers weten zelf zo min mogelijk, maar weten wel waar ze informatie kunnen vinden
- ☐ De presentation layer weet alles van GUI's
- ☐ Managers delegeren persistentie naar de DAO's
- ☐ Op initiatief van de GUI, halen Managers entiteiten op via de DAO's die ze aan de presentation layer geven
- ☐ De presentation layer gebruikt deze entiteiten om de GUI te renderen

DATA STORAGE LAYER

- ❑ Per persistente domein klasse: een DAO klasse
Naam <DomeinKlasseNaam>DAO
- ❑ DAO klasse bevat specifieke SQL statements die uitgevoerd worden door object van `DatabaseConnection` klasse

OPDRACHT

- ❑ Pas het ontwerp van de bibliotheek aan door de Data Storage Layer voor persistentie toe te voegen.
- ❑ Ga daarna aan de slag om de code aan te passen zodat deze overeenkomt met je ontwerp.

Maak DAO's voor 3 domeinklassen,

één voor Member die functioneel data aanlevert

Twee overige domeinklassen, mag met 'lege' methoden.

DISCUSSIE

☐ Hoe heb je de Data Storage laag toegevoegd?

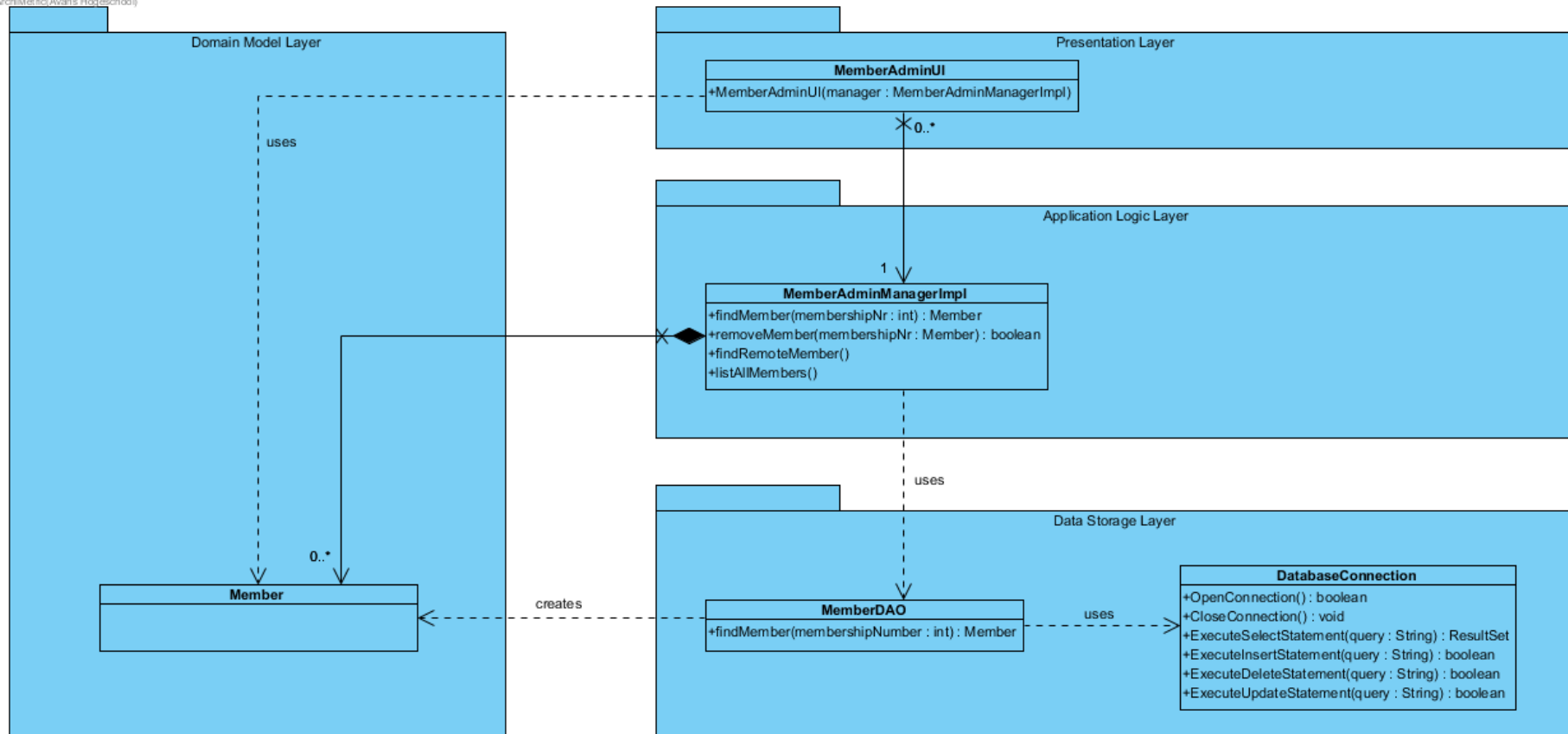
DATA STORAGE LAYER - ORM

- ❑ Eigenlijk is het maken van een data storage layer een bijna mechanisch proces

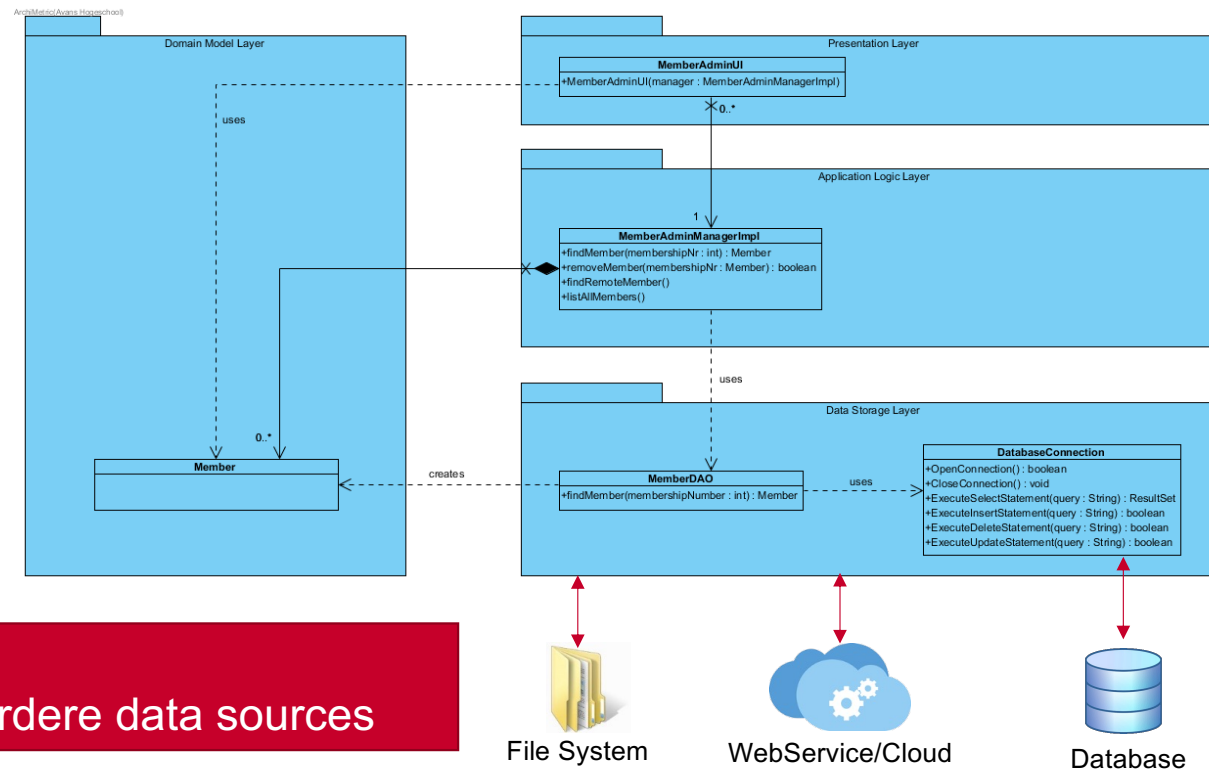
- ❑ Kan dus heel goed geautomatiseerd worden:
 - Object-relational mapping (ORM) tool
 - Voorbeelden: JPA, Hibernate, Entity Framework (P2.1)
 - Een DAO heet dan een CRUD repository

DATA STORAGE LAYER

Architectuur (Avans Hogeschool)



FLEXIBILITEIT: DATASTORAGE LAYER



Verandering:
Meerdere data sources

Wat verandert er in je ontwerp?

OPTIES VOOR HET ONTWERP

- De `MemberDAO` klasse aanpassen
 - Zodat deze meerdere bronnen aankan
 - Maar: Hoe wordt bepaald welke bron ‘actief is’, de klasse krijgt meerdere verantwoordelijkheden
- Per bron een `MemberDAO` variant
 - Klinkt goed, op voorwaarde dat:
de `MemberAdminManager` klasse er niets van merkt!

Dat kan met het ontwerpprincipe:

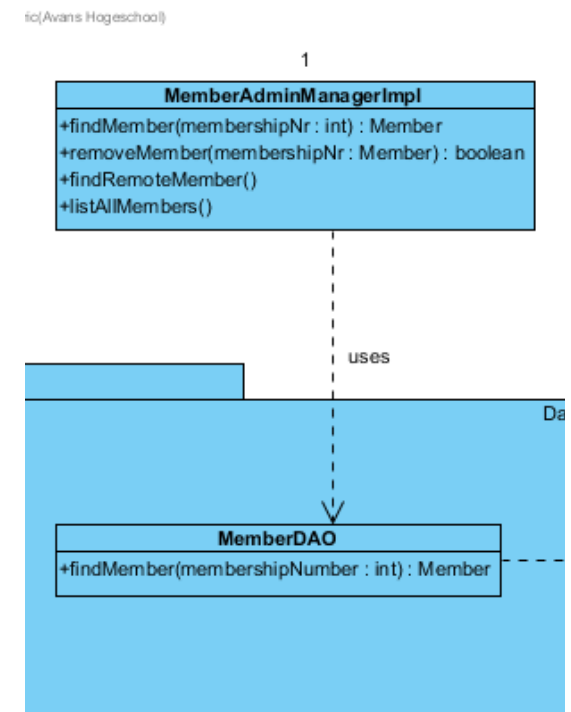
Program to an interface, instead of an implementation

PROGRAM TO AN INTERFACE

❑ `MemberAdminManager` is sterk gekoppeld aan de `MemberDAO`

❑ Toevoegen DAO-varianten op dezelfde wijze
vraagt om aanpassing `MemberAdminManager`

❑ Oplossing: **interface**



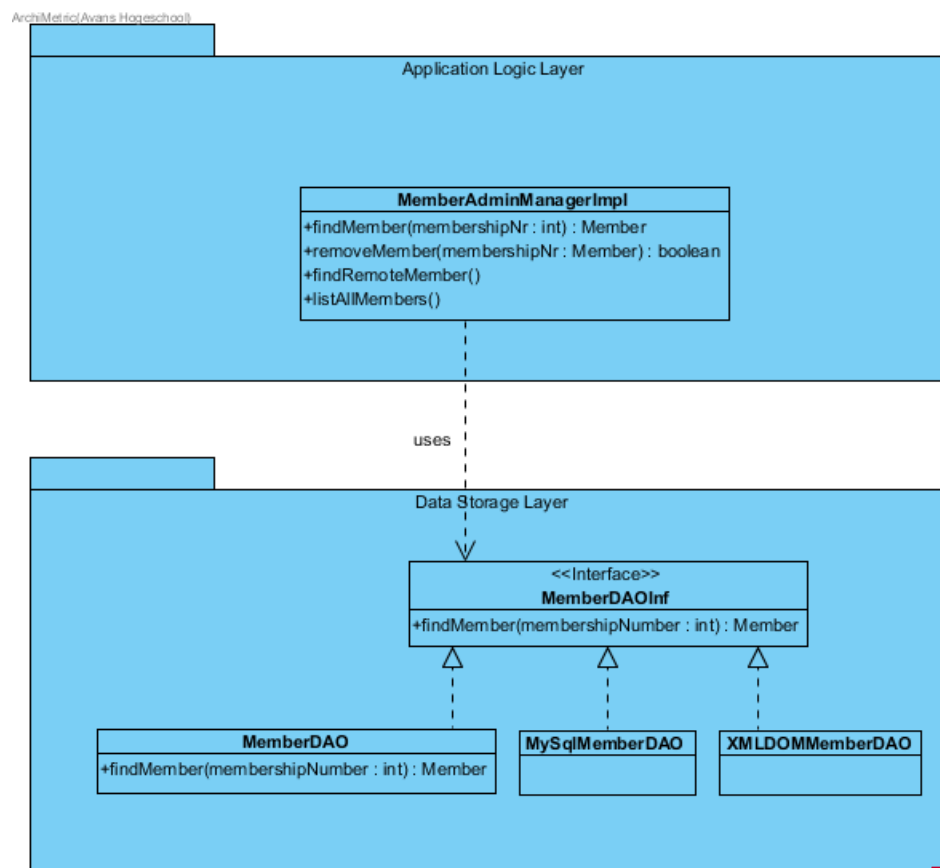
OPLOSSING MET INTERFACES

❑ De `MemberAdminManager` is nu onafhankelijk van de implementatie

❑ Implementatie wordt afgeschermd door de interface

De kleine lettertjes:

We accepteren voor nu dat de `MemberAdminManager` objecten maakt van de concrete DAO klassen en daarbij dus zelf de bron (database, webservice etc.) kiest



HUISWERK

Zie opdrachtomschrijving op Brightspace

Deadline: begin volgende les

avans
hogeschool



***VOLGENDE
KEER***

**Volgende keer
gaan we werken
loose coupling
en het factory
pattern**